DocuSense

Technical and architecture guide

DocuSense is a local document intelligence prototype built to demonstrate the full path from raw PDFs to measured retrieval and evidence-grounded generation. It combines data engineering, pretrained ML inference, exploratory data science, and application engineering. It does not train a foundation model or claim production readiness.

1. Architecture and lifecycle

The Streamlit interface manages a session-local library. PDF bytes are parsed with PyMuPDF; document hashes identify duplicate content. Page-bounded word windows preserve filename, page, offsets, and source text. SearchIndex accepts either a MiniLM encoder or an explicitly labeled TF-IDF baseline. Retrieval returns typed Hit records that both the UI and evaluation module can consume.

The generation module receives only the selected passages and question. It calls the Ollama HTTP endpoint on loopback, then validates every bracketed source label before displaying a generated answer. The analytics module calculates corpus statistics and performs exploratory K-Means and PCA using the same dense embeddings. Evaluation consumes verified page or chunk labels and exports query-level measurements.

The separation of pure core modules from the UI makes ranking, extraction, metrics, and unavailable-model behavior testable without launching a browser. Pretrained-model integration tests are separate from deterministic unit tests because weights and Ollama may be unavailable in CI.

2. Data model and provenance

Document ID is SHA-256 of the PDF bytes. Renaming the file preserves this ID, but resaving the PDF may change it even if displayed text looks similar. Deduplication uses this hash. Page IDs combine document ID and a one-based physical PDF page number; they do not use a printed page label from inside the paper.

Chunk ID is a truncated SHA-256 over document ID, page number, size, overlap, start offset, and passage text. It remains stable for identical content and configuration. Changing chunk settings changes IDs, so chunk-level labels must be regenerated. Page-level labels survive chunk-setting changes for identical PDF bytes.

Word windows never cross page boundaries. For size 160 and overlap 30, the stride is 130 words. The final window stops at the page end without generating redundant tail windows after the end has already been covered. Word length is not tokenizer length; MiniLM may truncate technical text that exceeds its token limit. Smaller windows or tokenizer-aware chunking are sensible extensions.

3. Embeddings and retrieval

The semantic encoder is sentence-transformers/all-MiniLM-L6-v2, running on CPU. It maps text into pretrained 384-dimensional representations. DocuSense uses these pretrained weights; it does not fine-tune them. Encoded passages are cached in a bounded session-local cache keyed by model name and text hash.

For vectors q and d, cosine similarity is dot(q, d) / (norm(q) * norm(d)). Normalize each vector by its L2 norm, then rank matrix @ query. Zero vectors remain finite and score zero. Scores lie approximately in [-1, 1] and are not calibrated relevance probabilities. Stable tie handling makes deterministic examples reproducible.

TF-IDF is an explicit lexical baseline, using English stopword removal and sublinear term frequency. It only returns positive-overlap results. Its sparse dot products are cosine similarities because the vectorizer normalizes rows. Dense scores and TF-IDF scores are not directly comparable calibration scales.

Dense retrieval can recover paraphrases but may struggle with specialized identifiers. TF-IDF is fast and interpretable, yet depends on vocabulary overlap. Comparing both on fixed labels is more informative than claiming dense retrieval is always superior.

4. Local generation and evidence checks

Ollama serves qwen2.5:1.5b at http://127.0.0.1:11434/api/generate. DocuSense sends a system instruction separately from JSON-serialized evidence. Temperature is zero and seed is 42. Generation is bounded to 450 predicted tokens with an 8192-token context configuration. Hardware and runtime differences can still change outputs.

Source labels S1 through Sk identify only the retrieved passages for that request. The generator uses an Ollama JSON schema containing individual claims, each with text and source labels. An empty claim list means abstention. The validator accepts exact labels and withholds malformed responses, unknown references, or uncited claims. The UI renders citations from the model-selected labels; it does not assign evidence to uncited claims automatically. Keeping one claim-list contract avoids contradictory answer and abstention fields in small model outputs.

Valid source IDs prove provenance membership, not entailment. A model may cite a real passage and misinterpret it. Prompt injection, incomplete evidence, extraction errors, and hallucinations remain possible. The user can inspect each full passage and page reference. Do not use this prototype for high-stakes decisions.

If Ollama is unreachable, the model is missing, the response is malformed, or a request times out, DocuSense displays evidence and a setup message. It does not synthesize an answer to conceal the failure. Retrieval and generation attempt latencies are measured separately with a monotonic clock.

5. Analytics and exploratory ML

Statistics use actual extracted pages and words, not overlapping chunk totals. Overlap can inflate term counts if analytics use chunks; document-level word frequencies therefore come from original page text. Keywords are lowercased alphabetic terms with English stopwords removed, which is a simple descriptive method and is not multilingual topic modeling.

K-Means runs on normalized dense embeddings with random_state=42 and ten initializations. Valid cluster counts range from two to the smaller of distinct embeddings and chunks minus one. A nearest-centroid passage represents each cluster. Frequency keywords provide interpretable descriptions, not ground-truth classes.

PCA projects the original embeddings into two dimensions for visualization. Clustering still happens in the original representation space. The projection can distort separation. Cosine silhouette is computed only when valid, with at most 200 points sampled per cluster. Stratified sampling prevents a tiny cluster from disappearing, but the result is a sample estimate.

6. Retrieval evaluation

Every Example has a question, a relevance unit, and a nonempty set of verified IDs. Labels referencing a different corpus or changed chunks are rejected before scoring. Recall@K is the distinct relevant IDs in the first K results divided by all annotated relevant IDs. The UI reports K=1,3,5. Several chunks from one page occupy actual ranks but count only once as relevant for page-level recall.

MRR@5 is the mean of the reciprocal rank of the first relevant result among five retrieved chunks; zero is assigned if none is found. The cutoff is stated explicitly. Metrics are macro averaged across questions, so each question has equal weight. Latency includes query encoding and ranking but excludes index construction and model loading.

The original demo has three educational PDFs, nine pages, and eighteen development questions. The annotation review file maps each question to its answer-bearing page. These narrow labels are not exhaustive judgments about every related passage. They are useful for reproducibility and ablation, but insufficient to establish real-world generalization.

The benchmark varies chunk size/overlap across 80/15, 160/30, and 220/40. With --semantic, it compares both engines on the same page labels. Index timing can benefit from a warm passage cache; do not interpret it as a controlled cold-start benchmark. Query encoding runs on every search so repeated questions do not become cache lookups. Query-level CSVs permit inspection of individual failures rather than relying only on averages.

The analysis script computes 5,000 paired bootstrap resamples of question-level MRR differences with seed 42. Percentile intervals describe sensitivity to the included questions. The small authored set and related questions limit independence; these intervals do not establish real-world generalization or causality. The notebook supports inspection of measured results and error queries.

Retrieval relevance and answer quality are separate evaluations. Add human-reviewed claim support, citation entailment, abstention correctness, and an independent test corpus before making generation-quality claims. No synthetic answer-quality score is included.

7. Privacy and operational limits

User PDF bytes, extracted text, labels, embeddings, and question history live in the Streamlit session. The app does not persist them automatically. Manifests and CSV downloads intentionally export text or questions when the user requests them. Model weights persist in .cache/huggingface, which is ignored by Git.

The server binds to loopback. This is a single-user local prototype with no authentication, authorization, OCR, malware scanner, or multi-tenant isolation. PDF parsing limits are 25 MB and 500 pages per file; the index is limited to 10,000 chunks. These limits are safeguards for local resource use, not comprehensive denial-of-service protection.

First setup needs internet for packages and model weights. Afterwards, inference can run locally once all required weights are available. Do not claim first-time offline setup. The app uses no paid APIs or API keys.

8. Engineering extensions

For a larger corpus, evaluate approximate nearest-neighbor indexing and versioned vector artifacts. Add hybrid retrieval and a reranker only after measuring baseline weaknesses. For production, introduce authentication, access controls, encrypted storage, document lifecycle policies, parsing isolation, resource quotas, observability, and model/version rollout controls.

Improve scientific validity with a larger independently annotated corpus, held-out questions, multiple annotators, label agreement analysis, confidence intervals, and error slices by document type or question difficulty. Add OCR with extraction-quality scoring for scanned PDFs.

9. Primary references

Sentence Transformers semantic similarity: https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html

Ollama generation API: https://docs.ollama.com/api/generate

MiniLM model card: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

PyMuPDF documentation: https://pymupdf.readthedocs.io/

Suggested research corpus: Attention Is All You Need, https://arxiv.org/abs/1706.03762 ; BERT, https://arxiv.org/abs/1810.04805 ; Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, https://arxiv.org/abs/2005.11401 . These papers are not redistributed with this repository.